

How can we detect cheating effectively without compromising system security through kernel-level access?

Gilles MEUNIER

École Publique d'Ingénieurs de la Santé Et du Numérique (EPISEN)
Université de Paris-Est Créteil (UPEC)



Presentation Outline

- 1 Context and Problem
- 2 Structural Limits of Kernel-Level Anti-Cheat
- 3 Server-Side Gameplay Analytics Pipeline
- 4 Conclusion

Outline

- 1 Context and Problem
- 2 Structural Limits of Kernel-Level Anti-Cheat
- 3 Server-Side Gameplay Analytics Pipeline
- 4 Conclusion

Context

- ▶ Competitive FPS integrity: same rules, same information, for everyone
- ▶ High stakes: \$75M prize pool at EWC 2026 (Paris), professional careers, ranked progression
- ▶ Cheats (aimbots, wallhacks) break this fairness

Context

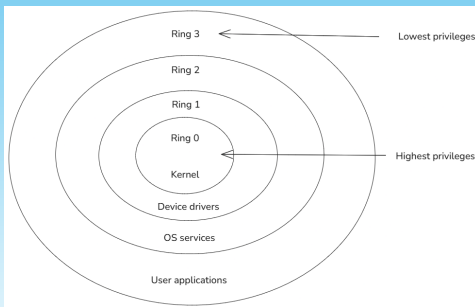
- ▶ Competitive FPS integrity: same rules, same information, for everyone
- ▶ High stakes: \$75M prize pool at EWC 2026 (Paris), professional careers, ranked progression
- ▶ Cheats (aimbots, wallhacks) break this fairness

Context

- ▶ Competitive FPS integrity: same rules, same information, for everyone
- ▶ High stakes: \$75M prize pool at EWC 2026 (Paris), professional careers, ranked progression
- ▶ Cheats (aimbots, wallhacks) break this fairness

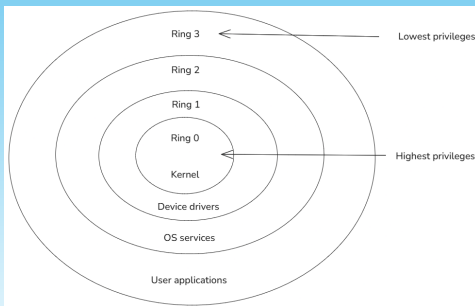
The industry's answer, and its cost

- ▶ Anti-cheat moves to Ring 0: full visibility over memory and processes
- ▶ Cost: a third-party driver gets unrestricted access to the entire machine
- ▶ Risks: cybersecurity, system stability, user privacy



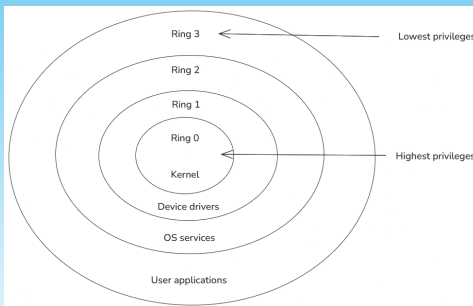
The industry's answer, and its cost

- ▶ Anti-cheat moves to Ring 0: full visibility over memory and processes
- ▶ Cost: a third-party driver gets unrestricted access to the entire machine
- ▶ Risks: cybersecurity, system stability, user privacy



The industry's answer, and its cost

- ▶ Anti-cheat moves to Ring 0: full visibility over memory and processes
- ▶ Cost: a third-party driver gets unrestricted access to the entire machine
- ▶ Risks: cybersecurity, system stability, user privacy



Problem

How can we detect cheating effectively without compromising system security through kernel-level access?

Outline

- 1 Context and Problem
- 2 Structural Limits of Kernel-Level Anti-Cheat
- 3 Server-Side Gameplay Analytics Pipeline
- 4 Conclusion

The arms race

- ▶ Ring 0 anti-cheat forces cheat developers to match that privilege level: does not solve the problem, only moves it
 - ▶ BYOVD: exploiting legitimate signed drivers to inject malicious code
 - ▶ Asymmetry: anti-cheat needs rigorous validation to ship, evasion techniques ship in days
- ⇒ A race the defender cannot win

The arms race

- ▶ Ring 0 anti-cheat forces cheat developers to match that privilege level: does not solve the problem, only moves it
 - ▶ BYOVD: exploiting legitimate signed drivers to inject malicious code
 - ▶ Asymmetry: anti-cheat needs rigorous validation to ship, evasion techniques ship in days
- ⇒ A race the defender cannot win

The arms race

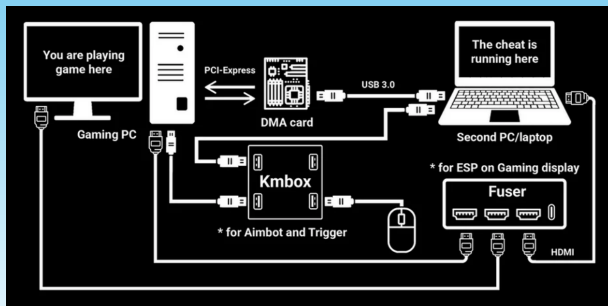
- ▶ Ring 0 anti-cheat forces cheat developers to match that privilege level: does not solve the problem, only moves it
 - ▶ BYOVD: exploiting legitimate signed drivers to inject malicious code
 - ▶ Asymmetry: anti-cheat needs rigorous validation to ship, evasion techniques ship in days
- ⇒ A race the defender cannot win

The arms race

- ▶ Ring 0 anti-cheat forces cheat developers to match that privilege level: does not solve the problem, only moves it
 - ▶ BYOVD: exploiting legitimate signed drivers to inject malicious code
 - ▶ Asymmetry: anti-cheat needs rigorous validation to ship, evasion techniques ship in days
- ⇒ A race the defender cannot win

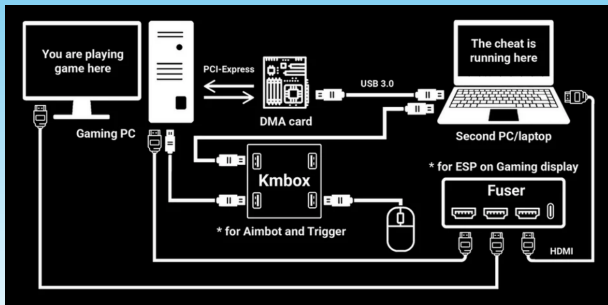
The hardware blind spot

- ▶ DMA cheats: a PCIe card reads host RAM directly, at the hardware layer
- ▶ Processing offloaded to a second PC — nothing runs on the gaming machine
- ▶ No driver, no process, no software footprint $\Rightarrow$ invisible to Ring 0 anti-cheat



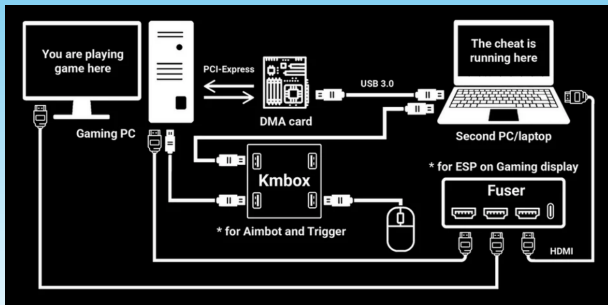
The hardware blind spot

- ▶ DMA cheats: a PCIe card reads host RAM directly, at the hardware layer
- ▶ Processing offloaded to a second PC — nothing runs on the gaming machine
- ▶ No driver, no process, no software footprint $\Rightarrow$ invisible to Ring 0 anti-cheat



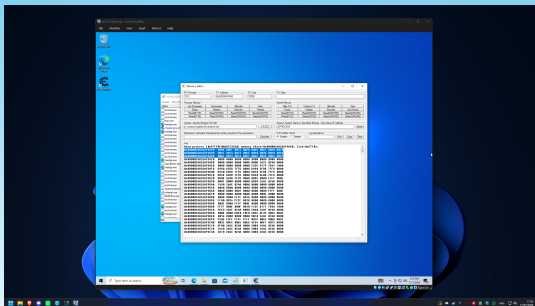
The hardware blind spot

- ▶ DMA cheats: a PCIe card reads host RAM directly, at the hardware layer
- ▶ Processing offloaded to a second PC — nothing runs on the gaming machine
- ▶ No driver, no process, no software footprint $\Rightarrow$ invisible to Ring 0 anti-cheat



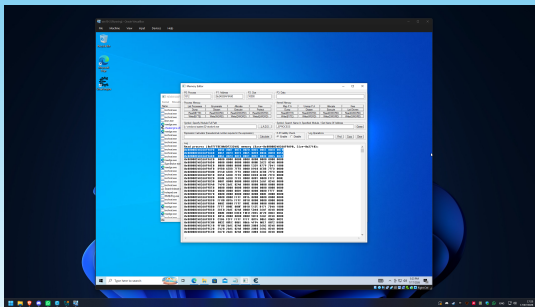
Experimental demonstration

- ▶ Setup: isolated Windows 10 VM, Notepad seeded with a password, Windows-Kernel-Explorer + Cheat Engine
- ▶ Result: password located and reconstructed from RAM in minutes, with zero cooperation from the target process
- ⇒ Same inspection power a Ring 0 driver already has, by design, over every background process



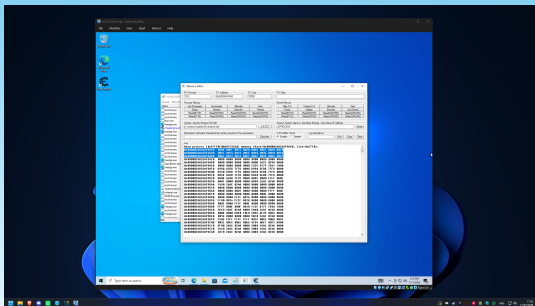
Experimental demonstration

- ▶ Setup: isolated Windows 10 VM, Notepad seeded with a password, Windows-Kernel-Explorer + Cheat Engine
 - ▶ Result: password located and reconstructed from RAM in minutes, with zero cooperation from the target process
- ⇒ Same inspection power a Ring 0 driver already has, by design, over every background process



Experimental demonstration

- ▶ Setup: isolated Windows 10 VM, Notepad seeded with a password, Windows-Kernel-Explorer + Cheat Engine
 - ▶ Result: password located and reconstructed from RAM in minutes, with zero cooperation from the target process
- ⇒ Same inspection power a Ring 0 driver already has, by design, over every background process



Problem

If detection at this privilege level is both:

- ▶ Intrusive: capable of reading anything on the machine
- ▶ Bypassable: outpaced by DMA cheats

Problem

If detection at this privilege level is both:

- ▶ Intrusive: capable of reading anything on the machine
- ▶ Bypassable: outpaced by DMA cheats

Is there a way to detect cheating without it?

Outline

- 1 Context and Problem
- 2 Structural Limits of Kernel-Level Anti-Cheat
- 3 Server-Side Gameplay Analytics Pipeline
- 4 Conclusion

Shifting the trust anchor

- ▶ Server is the authoritative source of match state: the client cannot tamper with it
- ▶ Analyze gameplay telemetry the server already receives, instead of inspecting the client
- ▶ No privileged access to the player's machine required

Shifting the trust anchor

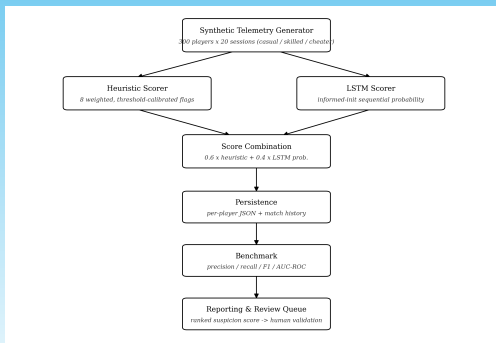
- ▶ Server is the authoritative source of match state: the client cannot tamper with it
- ▶ Analyze gameplay telemetry the server already receives, instead of inspecting the client
- ▶ No privileged access to the player's machine required

Shifting the trust anchor

- ▶ Server is the authoritative source of match state: the client cannot tamper with it
- ▶ Analyze gameplay telemetry the server already receives, instead of inspecting the client
- ▶ No privileged access to the player's machine required

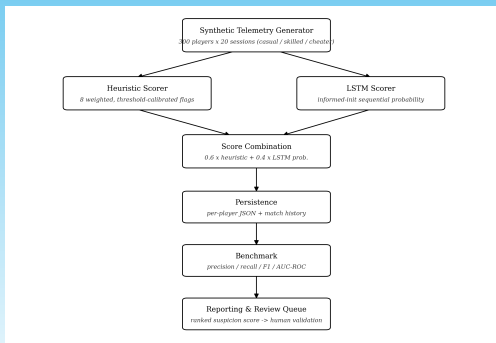
Pipeline architecture

- ▶ Threshold scorer: 8 flags (time-to-damage, headshot ratio, reaction time, etc)
- ▶ LSTM scorer: tracks anomaly consistency across rounds and matches
- ▶ Suspicion score: weighted combination $\Rightarrow$ human review



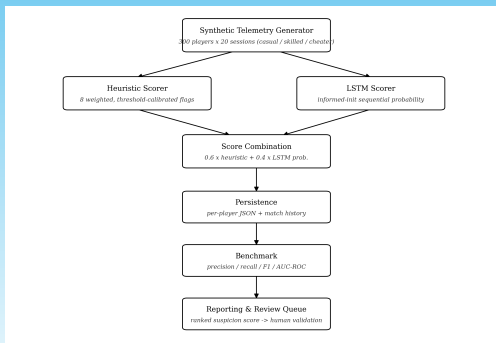
Pipeline architecture

- ▶ Threshold scorer: 8 flags (time-to-damage, headshot ratio, reaction time, etc)
- ▶ LSTM scorer: tracks anomaly consistency across rounds and matches
- ▶ Suspicion score: weighted combination $\Rightarrow$ human review



Pipeline architecture

- ▶ Threshold scorer: 8 flags (time-to-damage, headshot ratio, reaction time, etc)
- ▶ LSTM scorer: tracks anomaly consistency across rounds and matches
- ▶ Suspicion score: weighted combination $\Rightarrow$ human review



Dataset design and the high-skill bias

- ▶ Synthetic dataset: 300 players with casual / skilled / cheater profiles
- ▶ Calibrated from online benchmarks and personal experience
- ▶ Challenge: elite players can statistically resemble subtle cheaters
- ▶ Distributions deliberately overlapped around that boundary to empathize the hardest case



Dataset design and the high-skill bias

- ▶ Synthetic dataset: 300 players with casual / skilled / cheater profiles
- ▶ Calibrated from online benchmarks and personal experience
- ▶ Challenge: elite players can statistically resemble subtle cheaters
- ▶ Distributions deliberately overlapped around that boundary to empathize the hardest case



Dataset design and the high-skill bias

- ▶ Synthetic dataset: 300 players with casual / skilled / cheater profiles
- ▶ Calibrated from online benchmarks and personal experience
- ▶ Challenge: elite players can statistically resemble subtle cheaters
- ▶ Distributions deliberately overlapped around that boundary to empathize the hardest case



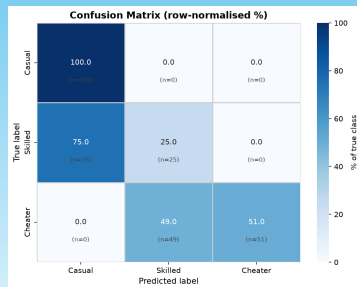
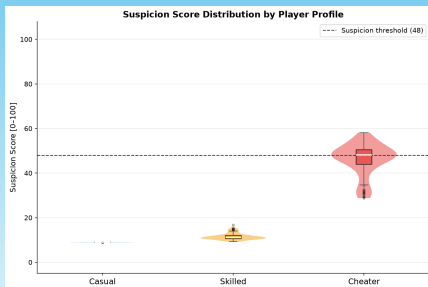
Dataset design and the high-skill bias

- ▶ Synthetic dataset: 300 players with casual / skilled / cheater profiles
- ▶ Calibrated from online benchmarks and personal experience
- ▶ Challenge: elite players can statistically resemble subtle cheaters
- ▶ Distributions deliberately overlapped around that boundary to empathize the hardest case



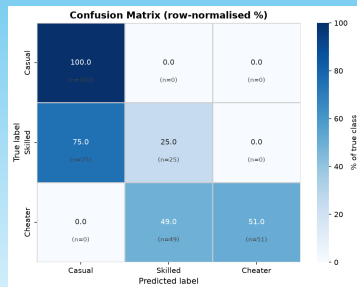
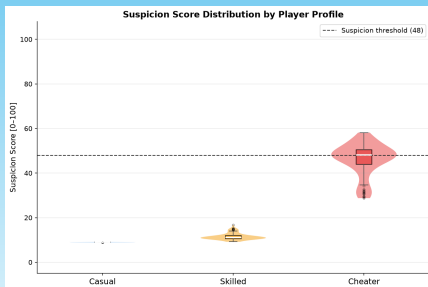
Results

- ▶ Clear statistical separation between legitimate players and cheaters at the population level
- ▶ 0% false positives on legitimate players, 51% of cheaters correctly flagged



Results

- ▶ Clear statistical separation between legitimate players and cheaters at the population level
- ▶ **0% false positives on legitimate players, 51% of cheaters correctly flagged**



Outline

- 1 Context and Problem
- 2 Structural Limits of Kernel-Level Anti-Cheat
- 3 Server-Side Gameplay Analytics Pipeline
- 4 Conclusion

Limitations

- ▶ Dataset and LSTM layer are entirely synthetic
- ▶ Flag thresholds are hardcoded instead of being trained
- ▶ Social features are not considered

Limitations

- ▶ Dataset and LSTM layer are entirely synthetic
- ▶ Flag thresholds are hardcoded instead of being trained
- ▶ Social features are not considered

Limitations

- ▶ Dataset and LSTM layer are entirely synthetic
- ▶ Flag thresholds are hardcoded instead of being trained
- ▶ Social features are not considered

Synthesis

- ▶ Server-side behavioral analysis: viable, effective, privacy-preserving alternative to intrusive client drivers
 - ▶ Removes the hardware blind spot created by DMA devices
 - ▶ Ring 0 drivers cannot be the future of anti-cheat, given their privacy cost
- ⇒ Defense-in-depth: lightweight client checks + server-side scoring + human review

Synthesis

- ▶ Server-side behavioral analysis: viable, effective, privacy-preserving alternative to intrusive client drivers
 - ▶ Removes the hardware blind spot created by DMA devices
 - ▶ Ring 0 drivers cannot be the future of anti-cheat, given their privacy cost
- ⇒ Defense-in-depth: lightweight client checks + server-side scoring + human review

Synthesis

- ▶ Server-side behavioral analysis: viable, effective, privacy-preserving alternative to intrusive client drivers
 - ▶ Removes the hardware blind spot created by DMA devices
 - ▶ Ring 0 drivers cannot be the future of anti-cheat, given their privacy cost
- ⇒ Defense-in-depth: lightweight client checks + server-side scoring + human review

Synthesis

- ▶ Server-side behavioral analysis: viable, effective, privacy-preserving alternative to intrusive client drivers
 - ▶ Removes the hardware blind spot created by DMA devices
 - ▶ Ring 0 drivers cannot be the future of anti-cheat, given their privacy cost
- ⇒ Defense-in-depth: lightweight client checks + server-side scoring + human review

Thank you for your attention!

Any questions?